ITE3162 – PYTHON FUNDAMENTALS

DR. ERIC HITIMANA

OUTLINES

- Introduction
- Data Types and Variables
- Operators
- Conditional Statements
- Python Data Structure
- Python Controls
- Python Functions
- OOP In Python
- Python Modules and Packages

INTRODUCTION

https://python.org

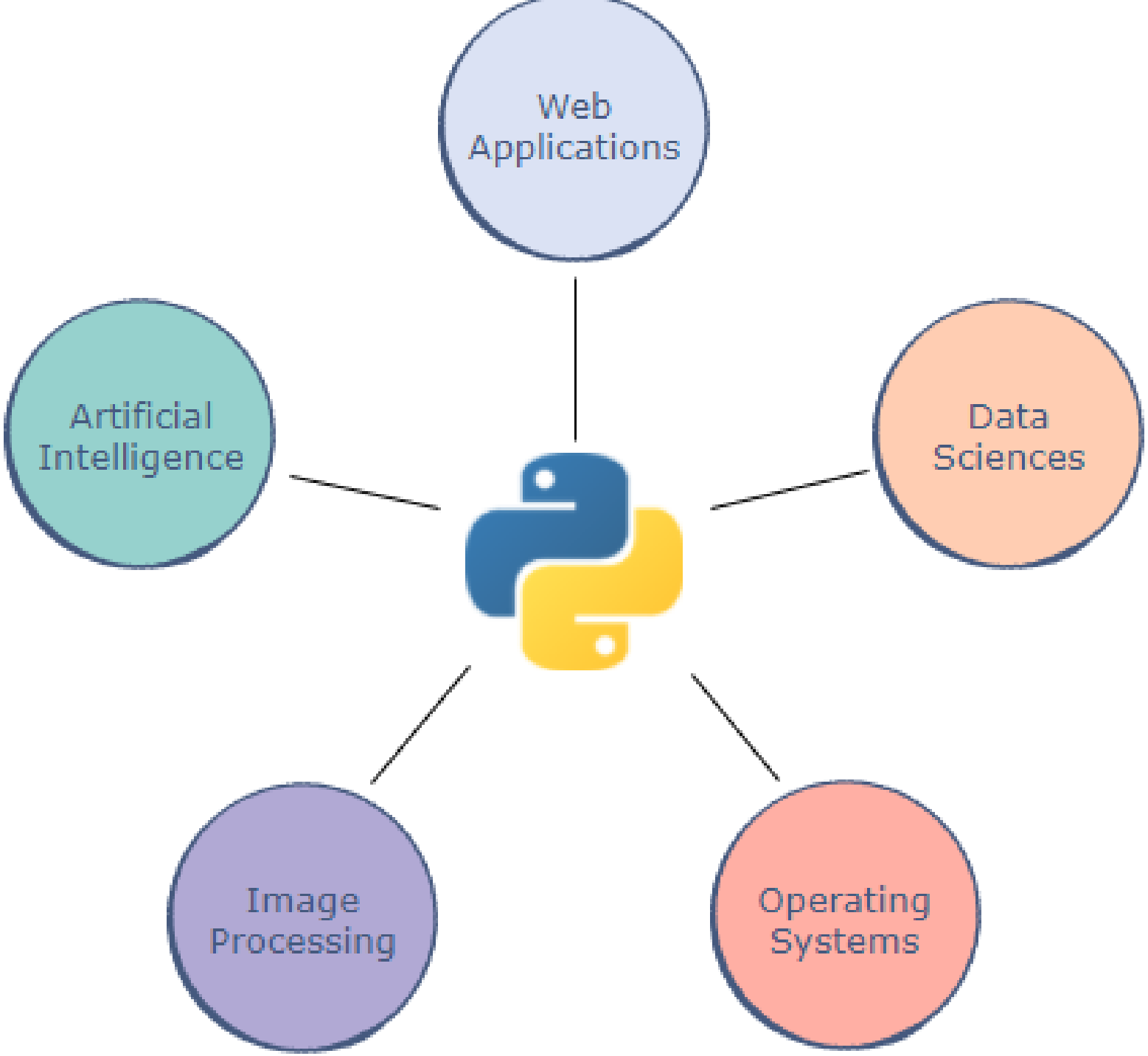
https://code.visualstudio.com/

WHAT IS PYTHON?

Developed in 1990, Python is one of the most popular general-purpose programming languages in modern times.

Characteristics of python:

- A High-level language
- Readability
- Applications



PYTHON VERSIONS

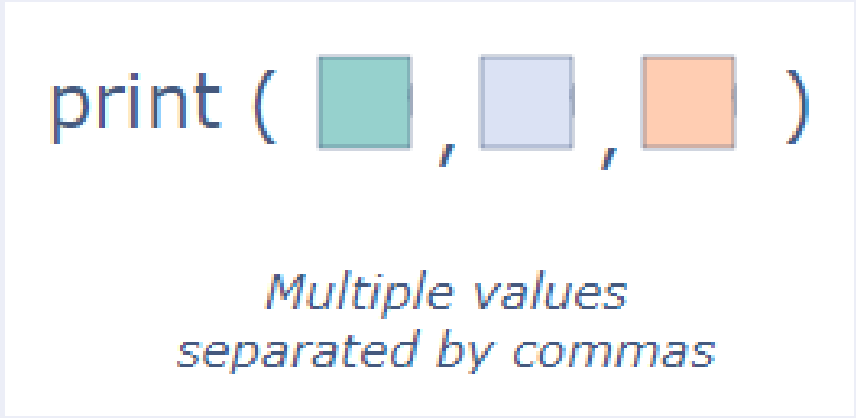
- Python has had several major updates in the past.
- Python 2.7 was widely used for a very long time, even after the release of newer versions
- Python 2.7 has been deprecated as of January 01 2020, and replaced completely by 3.xx
- To keep up with the latest technologies, we'll use Python 3 for this course.

WRITING OUR FIRST CODE - **THE PRINT STATEMENT**

- Every language has a different syntax for displaying or printing something on the screen. The Python file extension is **.py**
- We can print data on the terminal by simply using the print statement.
- print (data)
- E.g:
```
print("Hello World")
```
- Next, we'll print a few numbers. Each call to print moves the output to a new line:

```
print(50)
print(1000)
print(3.142)
```

PRINTING MULTIPLE PIECES OF DATA



Printing multiple things in a single print command; we must just separate them using **commas**

If we want multiple print statements to print in the same line, we use the "end" parameter

```
print("Hello", end="")
print("World")

print("Hello", end=" ")
print("World")
```

COMMENTS

- **Comments** are pieces of text used to describe what is happening in the code.
- A comment can be written using the **#** character:

```
print(50)  # This line prints 50
print("Hello World")  # This line prints Hello World

# This is just a comment hanging out on its own!

# For multi-line comments, we must
# add the hashtag symbol
# each time
```

- An alternative to these multi-line comments (line 4 - 8) are docstrings.
- They are encased in triple quotes, """

```
""" Docstrings are pretty cool
for writing longer comments
or notes about the code"""
```

EXAMPLE & EXERCISES

Self-check

1. What is the command to show the installed version of python?
2. How do you open vscode in your current directory
3. What do you think the following code will print " **print(Let's begin)** "
4. How do we run python programs in visual studio code?

Exercise, Task:

1. Use vscode to create a python file with your name
2. Write a two lines of code that outputs your name and the phrase "python is fun" on one line.
3. Execute the program and see the output

DATA TYPES AND VARIABLES

The data type of an item defines the type and range of values an item can have.

A variable is simply a name to which a value can be assigned.

PYTHON'S DATA TYPES

Unlike many other languages, Python does not strongly emphasize defining the data type of an object, which makes coding much simpler.

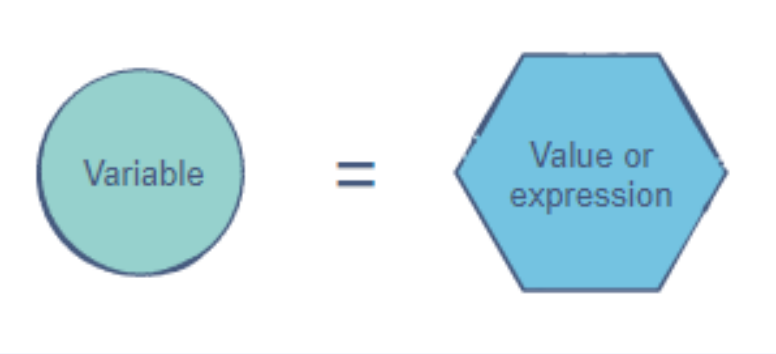
The language provides three main data types:

1. **Number**
2. **Strings**
3. **Booleans**

VARIABLES

Variables allow us to give meaningful names to data

The simplest way to assign a value to a variable is through the **=** operator



Variables are **mutable**

Variables allow us to store data so that we can use it later to perform operations in the code

NAMING CONVENTION

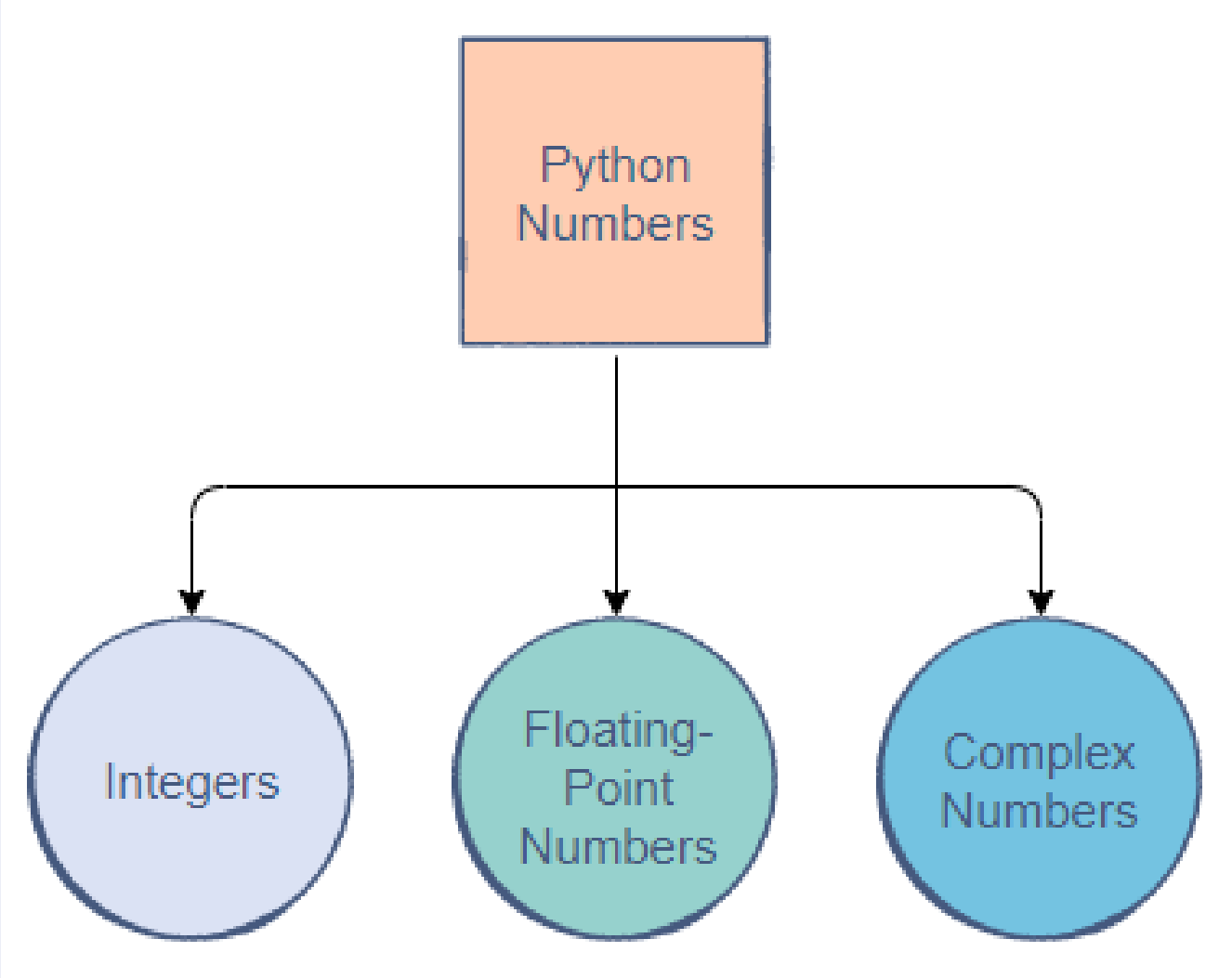
There are certain rules we have to follow when picking the name for a variable:

- The name can start with an upper- or lower-case alphabet.
- A number can appear in the name, but not at the beginning.
- The _ character can appear anywhere in the name.
- Spaces are not allowed. Instead, we must use like **snake_case** to make variable names readable.
- The name of the variable should be something meaningful that describes the value it holds, instead of being random characters.

1. NUMBERS

Python is one of the most powerful languages when it comes to manipulating numerical data

There are three main types of numbers in Python:



INTEGER NUMBERS

The integer data type is comprised of all the positive and negative whole numbers

```
print(10)   # A positive integer
print(-3000)   # A negative integer

num = 123456789   # Assigning an integer to a variable
print(num)
num = -16000   # Assigning a new integer
print(num)
```

Note: In Python, all negative numbers start with the - symbol.

FLOATING POINT NUMBERS

Floating-point numbers, or floats, refer to positive and negative decimal numbers

Python allows us to create decimals up to a very high decimal place

This ensures accurate computations for precise values.

```
print(1.00000000005)   # A positive float
print(-85.6701)   # A negative float

flt_pt = 1.23456789
print(flt_pt)
```

In Python, **5** is considered to be an integer while **5.0** is a float

COMPLEX NUMBERS

Python also supports complex numbers

Complex numbers are made up of a real and an imaginary part.

Just like the ***print()*** statement is used to print values, ***complex()*** is used to create complex numbers

complex(real, imaginary)

```
print(complex(10, 20))  # Represents the complex number (10 + 20j)
print(complex(2.5, -18.2))  # Represents the complex number (2.5 - 18.2j)

complex_1 = complex(0, 2)
complex_2 = complex(2, 0)
print(complex_1)
print(complex_2)
```

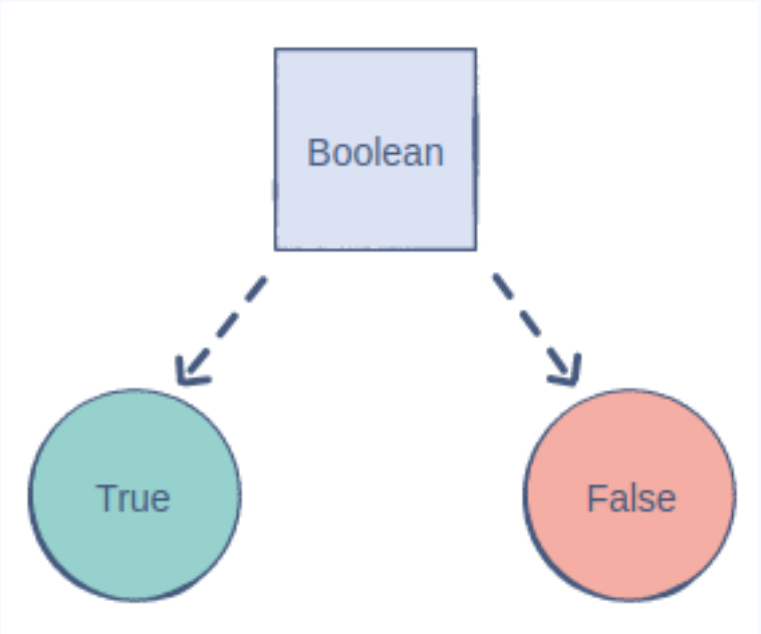
2. BOOLEANS

The Boolean (also known as bool) data type allows us to choose between two values: true and false

A Boolean is used to determine whether the logic of an expression or a comparison is correct.

It plays a huge role in data comparisons

```
1  print(True)
2
3  f_bool = False
4  print(f_bool)
5
```



3. STRINGS

A string is a collection of characters closed within single or double quotation marks.

A group of characters such as **"Hello World"** is an example of the string data type.

A string can also contain a single character or be empty

```
print("Harry Potter!")  # Double quotation marks

got = 'Game of Thrones...'  # Single quotation marks
print(got)
print("$")  # Single character

empty = ""
print(empty)  # Just prints an empty line

```

THE LENGTH OF A STRING

The length of a string can be found using the **len** statement

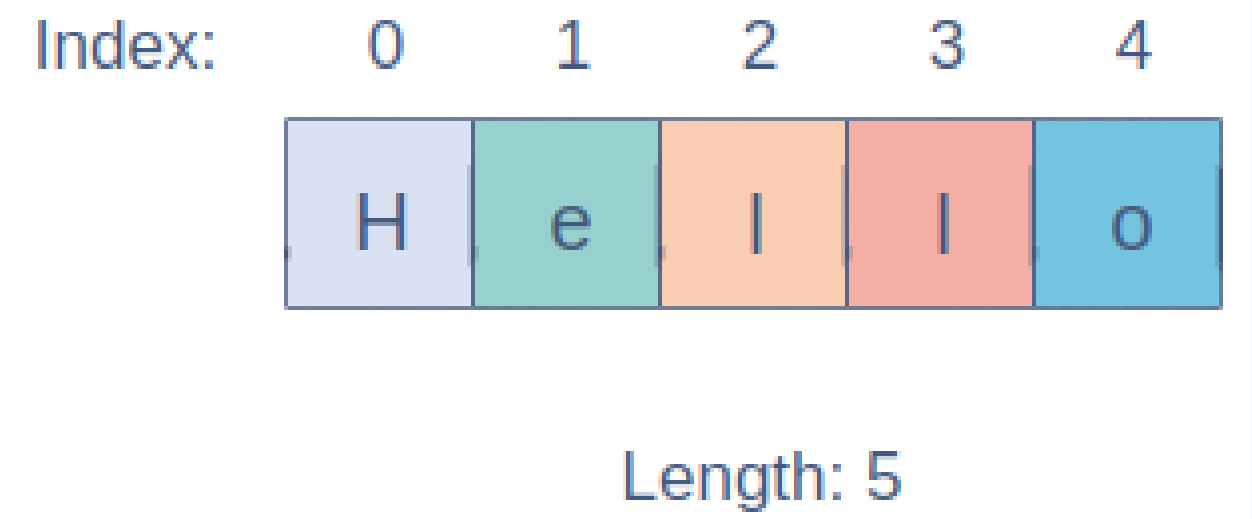
This length indicates the number of characters in the string:

```
1  random_string = "I am Batman"  # 11 characters
2  print(len(random_string))
3
```

STRING INDEXING

In a string, every character is given a numerical **index** based on its position.

A string in Python is indexed from 0 to n-1 where n is its length



MANIPULATING STRINGS

Accessing Characters:

- Each character in a string can be accessed using its index.
- The index must be closed within square brackets, [], and appended to the string

Reverse Indexing:

- We can also change our indexing convention by using negative indices
- Negative indices start from the opposite end of the string

String Slicing: Slicing is the process of obtaining a portion (substring) of a string by using its indices

```
string[start:end]
```

MANIPULATING STRINGS …..

Slicing with a Step:

- we can define a **step** through which we can skip characters in the string
- The default step is 1

```
string[start:end:step]
```

Reverse Slicing: Strings can also be sliced to return a reversed substring.

- In this case, we would need to switch the order of the **start** and **end** indices.

Partial Slicing: One thing to note is that specifying the **start** and **end** indices is optional.

If **start** is not provided, the substring will have all the characters until the **end** index.

- If **end** is not provided, the substring will begin from the **start** index and go all the way to the end

EXAMPLE & EXERCISES

Given the following variable
my_string = "This is My String !"
print(my_string[0])
Print(my_string[len(my_string)])
How would I print the last ! Mark in my_string variable
print(my_string[len(my_string) - 1])
print(my_string[-1])
print(my_string[1:7])
print(my_string[0:7:2])
print(my_string[13:2:-1])
Print(my_string[17:0:-2])
print(my_string[:])
print(my_string[:: -1])

GETTING USER INPUT

The Input is nothing but some value from a system or user.

In Python, we have the following two functions to handle input from a user and system

- **input(prompt)** to accept input from a user
- **print()** to display output on the console.

Example:

```
name = input("Enter Employee Name ")
salary = input("Enter salary ")
company = input("Enter Company name ")

print("\n")
print("Printing Employee Details")
print("Name", "Salary", "Company")
print(name, salary, company)
```

DATA TYPE CONVERSION

When programming, there are times we need to convert values between types in order to manipulate values in a different way

- **Converting Integers to Floats :** we use float() method
- **Converting Floats to Integers:** we use int() method
- **Converting Numbers to Strings:** we use str() method
- **Converting Strings to Numbers:** we use the int() and float() methods.

EXAMPLE & EXERCISES

1. Write a simple program that prints the total number of characters of a string entered by the user.

PYTHON OPERATORS

- **Operators** are used to perform arithmetic and log ical operations on data.
- They enable us to manipulate and interpret data to produce useful outputs."

OUTLINES

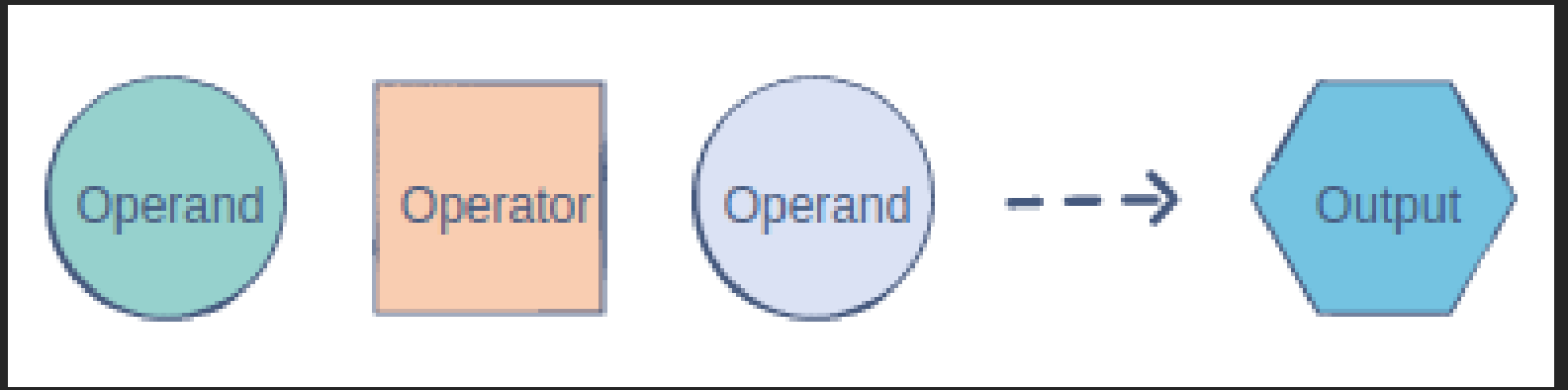
- Python Operators
- Conditional Statements

OPERATORS

Python operators follow the **in-fix** or **prefix** notations

The 5 main operator types in Python are:

- arithmetic operators
- comparison operators
- assignment operators
- logical operators
- bitwise operators



Operator	Purpose	Notation
`()`	Parentheses	Encapsulates the Precedent Operation
`**`	Exponent	In-fix
`%`, `*`, `/`, `//`	Modulo, Multiplication, Division, Floor Division	In-fix
`+`, `-`	Addition, Subtraction	In-fix

ARITHMETIC OPERATORS

THE OPERATOR LISTED HIGHER WILL BE COMPUTED FIRST

Operator	Purpose	Notation
`>`	Greater Than	In-fix
`<`	Less Than	In-fix
`>=`	Greater Than or Equal To	In-fix
`<=`	Less Than or Equal To	In-fix
`==`	Equal to	In-fix
`is`	Equal to	In-fix
`is not`	Not Equal To	In-fix

COMPARISON OPERATORS

CAN BE USED TO COMPARE VALUES IN MATHEMATICAL TERMS.

Operator	Purpose	Notation
`=`	Assign	In-fix
`+=`	Add and Assign	In-fix
`-=`	Subtract and Assign	In-fix
`*=`	Multiply and Assign	In-fix
`/=`	Divide and Assign	In-fix
`//=`	Divide, Floor, and Assign	In-fix
`**=`	Raise power and Assign	In-fix
`%=`	Take Modulo and Assign	In-fix
`|=`, `&=`, `^=`	OR/AND/XOR and Assign	In-fix
`>>=`	Right-shift and Assign	In-fix
`<<=`	Left-shift and Assign	In-fix

ASSIGNMENT OPERATORS

USED TO ASSIGN VALUES TO A VARIABLE.

Operator	Purpose	Notation
`and`	AND	In-fix
`or`	OR	In-fix
`not`	NOT	Prefix

LOGICAL OPERATORS

CAN BE USED TO COMPARE VALUES IN MATHEMATICAL TERMS.

LOGICAL EXPRESSIONS

Logical operators are used to manipulate the logic of *Boolean expressions*

```
1  # OR Expression
2  my_bool = True or False
3  print(my_bool)
4
5  # AND Expression
6  my_bool = True and False
7  print(my_bool)
8
9  # NOT expression
10 my_bool = False
11 print(not my_bool)
12
```

STRING OPERATIONS

The string data type has numerous utilities that make string computations much easier

Comparison Operators : Strings are compatible with the comparison operators

When two strings have different lengths, the string which comes first in the dictionary is said to have the smaller value.

Concatenation: The **+** operator can be used to merge two strings together

The ***** operator allows us to multiply a string, resulting in a repeating pattern

Search: The **in** keyword can be used to check if a particular substring exists in another string. If the substring is found, the operation returns **true**

EXAMPLE & EXERCISES

1. Assuming that variables have the following boolean values:

a = True
b = False
c = a and not b

Enter the result of evaluating the expression:

a and (not c or b)

Exercise

1. Please, print() the result of the following expression: raise **31** to the power of **331** and then get the remainder of the result's division by **20**.
2. What is the result of the following Python expression? **Print(((3 + 5) // 2 * 2 ** 3) % 7)**.
3. Find out if the result of dividing A by B is an odd number.

Sample Input 1: 99 & 3 **Sample Output 1:** True

EXERCISES

Exercise

The movie theater has cinema halls that can accommodate a certain number of viewers each. Figure out if a movie theater can hold a given number of viewers that plan to visit it on a particular day.

The input format

The first line is number of halls, the second line is their capacity, and the third line is the planned number of viewers.

The output format

True or False.

Sample Input 1:

9

68

589

Sample Output 1:

True

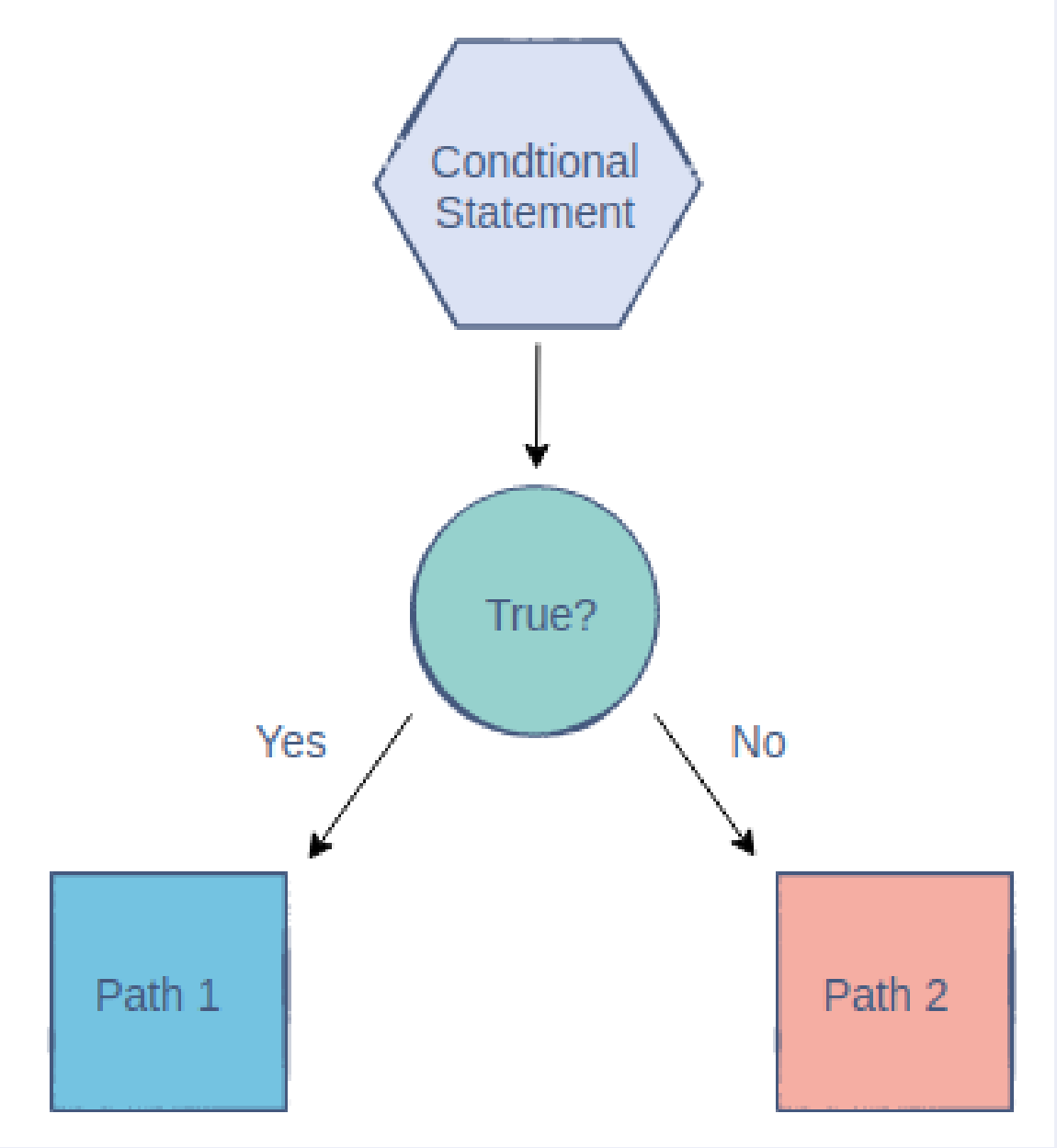
CONDITIONAL STATEMENTS

"A conditional statement is a Boolean expression that, if **True**, executes a piece of code".

CONDITIONAL STATEMENTS

It allows programs to branch out into different paths based on Boolean expressions result in **True** or **False** outcomes

conditional statements control the flow of the code and allow the computer to think



CONDITIONAL STATEMENTS IN PYTHON

There are three types of conditional statements in Python:

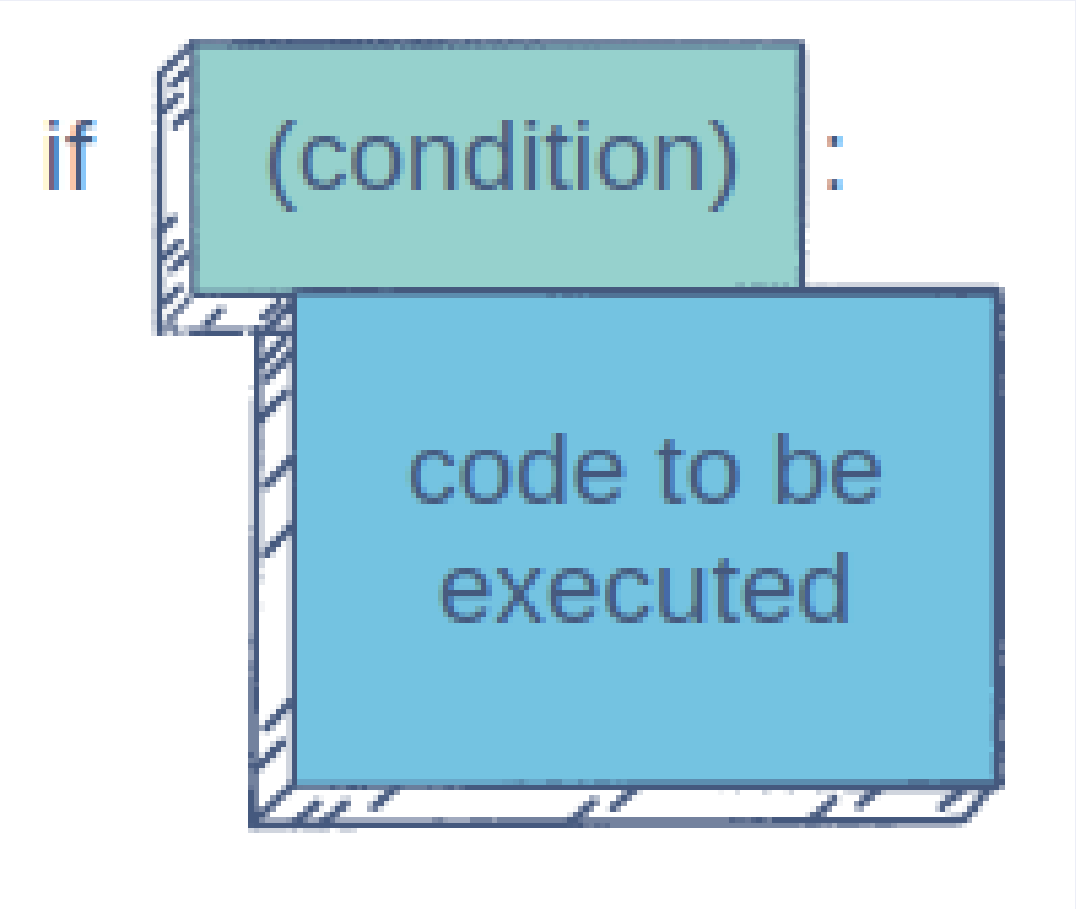
if

if-else

if-elif-else

```
1  if condtional statement is True:
2      # execute expression1
3      pass
4  else:
5      # execute expression2
6      pass
7
```

THE IF STATEMENT



The simplest conditional statement that we can write is the if statement. It comprises of two parts:

The **condition**

The **code to be executed**

CONDITIONS WITH LOGICAL OPERATORS

We can use logical operators to create more complex conditions in the **if** statement.

```
num = 12

if num % 2 == 0 and num % 3 == 0 and num % 4 == 0:
    # Only works when num is a multiple of 2, 3, and 4
    print("The number is a multiple of 2, 3, and 4")

if (num % 5 == 0 or num % 6 == 0):
    # Only works when num is either a multiple of 5 or 6
    print("The number is a multiple of 5 and/or 6")

```

NESTED IF STATEMENTS

A cool feature of conditional statements is that we can nest them.

This means that there could be an **if** statement inside another!

```
num = 63

if num >= 0 and num <= 100:
    if num >= 50 and num <= 75:
        if num >= 60 and num <= 70:
            print("The number is in the 60-70 range")

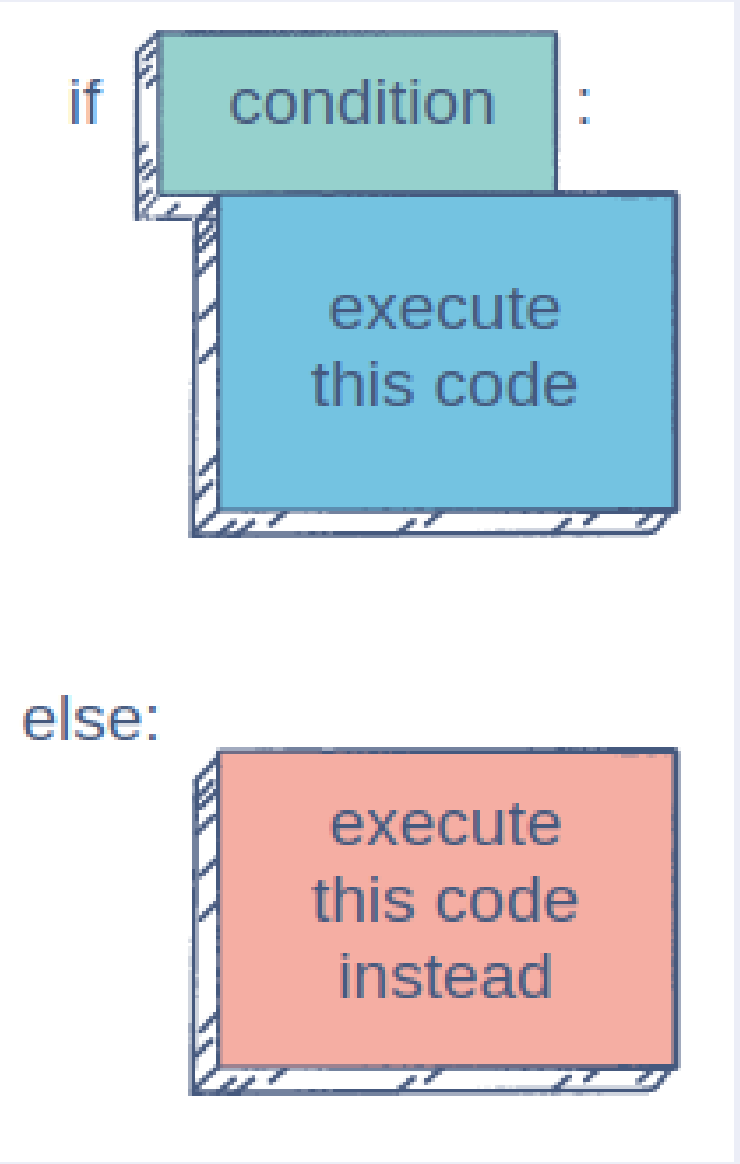
```

THE IF-ELSE STATEMENT

What if we wanted to execute a different set of operations in case the **if** condition turns out to be **False**?

The **else** keyword will be on the same indentation level as the **if** keyword. Its body will be indented one tab to the right just like the **if** statement.

```
1  num = 60
2
3  if num <= 50:
4      print("The number is less than or equal to 50")
5  else:
6      print("The number is greater than 50")
7
```



CONDITIONAL EXPRESSIONS

Conditional expressions use the functionality of an **if-else** statement in a different way

A conditional expression can be written in the following way:

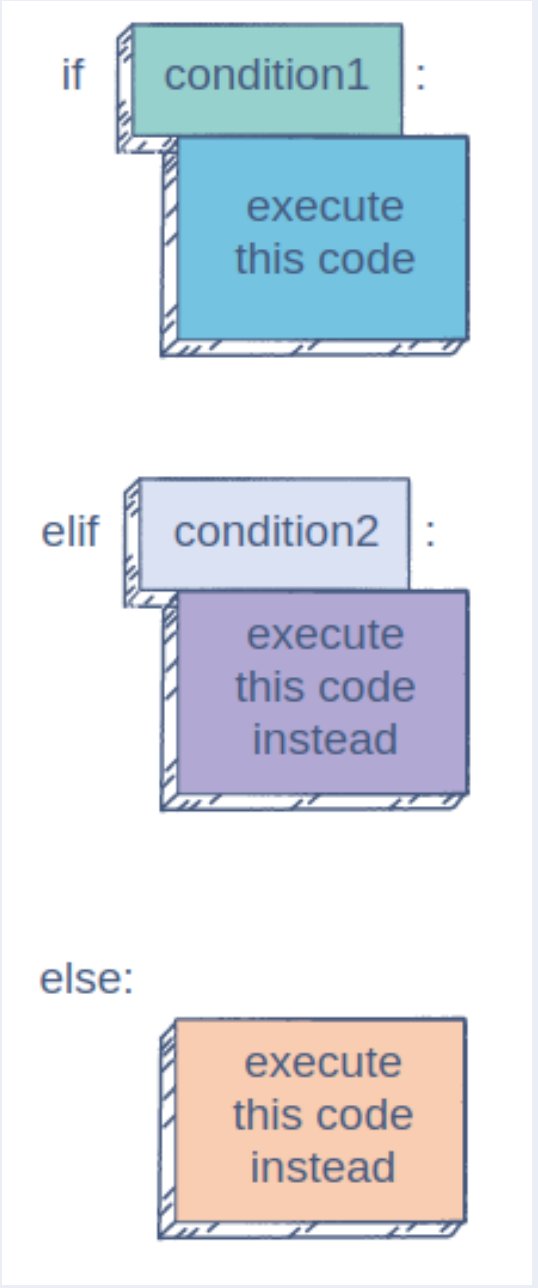
```
output_value1 if condition else output_value2
```

If the **if** condition is fulfilled, the output would be **output_value1**. Otherwise, it would be **output_value2.**

```
1  num = 60
2
3  output = "The number is less than or equal to 50" \
4      if num <= 50 else "The number is greater than 50"
5
6  print(output)
7
```

THE IF-ELIF-ELSE STATEMENT

It is the most comprehensive conditional statement because it allows us to create multiple conditions easily.



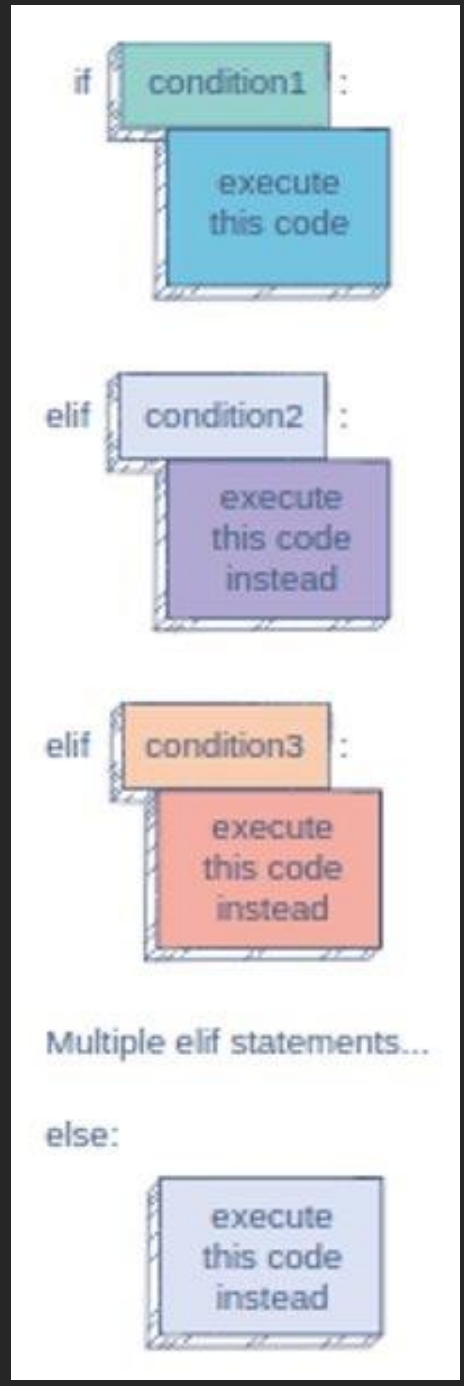
IF-ELIF EXAMPLE

```
1  light = "Red"
2
3  if light == "Green":
4      print("Go")
5
6  elif light == "Yellow":
7      print("Caution")
8
9  elif light == "Red":
10     print("Stop")
11
12 else:
13     print("Incorrect light signal")
14
```

MULTIPLE ELIF ST ATEMENTS

This is the beauty of the **if-elif-else** statement.

We can have as many **elifs** as we require, as long as they come between **if** and **else**



EXAMPLE & EXERCISES

1. **Discounted Price**

In this Exercise, you must discount a price according to its value.

- If the price is 300 or above, there will be a 30% discount.
- If the price is between 200 and 300 (200 inclusive), there will be a 20% discount.
- If the price is between 100 and 200 (100 inclusive), there will be a 10% discount.
- If the price is less than 100, there will be a 5% discount.
- If the price is negative, there will be no discount

Sample Input: price = 250

Sample Output: price = 200

EXAMPLE & EXERCISES

A digital system monitors the temperature of a processor. Write a Python program that reads the processor temperature and displays:

- **Below 50°C:** "Normal"
- **50°C–69°C:** "Warning"
- **70°C–84°C:** "High Temperature"
- **85°C or above:** "Critical Temperature - Shutdown Required"

PYTHON DATA STRUCTURES

- **LIST**
- **DICTIONARY**

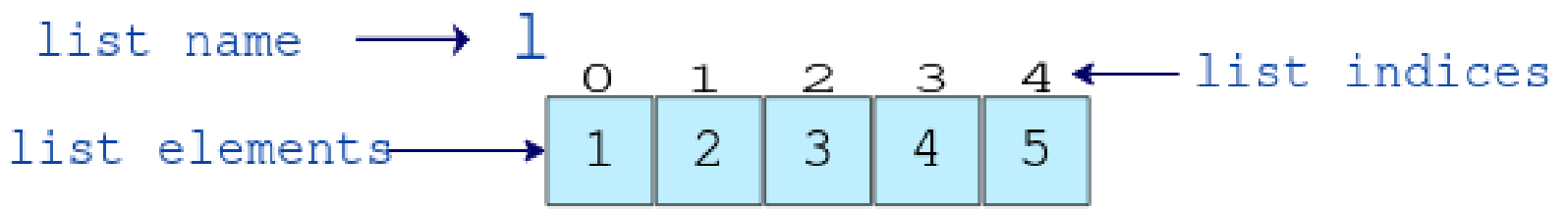
WHAT ARE LISTS ?

Python lists are data structures that group sequences of elements.

Lists **can have elements of several types**, and you can also **mix different types** within the same list (although all elements are usually of the same datatype).

Lists are created using square brackets [], and the elements are separated by commas (,).

The elements in a list can be accessed by their positions, starting with 0 as the index of the first element.



SUBLIST

```
sublist=list[startindex:endindex]
```

- Sometimes you want just a small portion of a list—a **sublist.**
- Sublists are simply subsets of a list; they can be retrieved using a technique called *slicing*.

Note: The start index is inclusive and end index is exclusive in the range.

OPERATIONS ON LIST

List Concatenation

- Lists can be concatenated using the **+** operator

Traverse a List

- Lists can be iterated over using for loops in Python

List Methods

Python List append()

- Add a single element to the end of the list [**list.append(item)**]

Python List clear()

- Removes all Items from the List [**list.clear()**]

```
students = ["Alice", "Bob"]
new_students = ["John", "Mary"]

# Concatenate
students = students + new_students
print(students)

# Traverse
for student in students:
    print(student)

# Append
students.append("David")
print(students)

# Clear
students.clear()
print(students)
```

LIST METHODS

Python List copy()

- returns a shallow copy of the list

Python List count()

- returns count of the element in the list [**list.count(element)**]

Python List extend()

- adds iterable elements to the end of the list [**list1.extend(iterable)**]

Python List index()

- returns the index of the element in the list [**list.index(element, start, end)**]

Python List insert()

- insert an element to the list [**list.insert(i, elem)**]

```python
# Create a list
students = ["Alice", "Bob", "John", "Bob"]

print("Original list:")
print(students)


# 1. copy() - Copy the list
students_copy = students.copy()

print("\nCopied list:")
print(students_copy)


# 2. count() - Count an element
bob_count = students.count("Bob")

print("\nNumber of times Bob appears:")
print(bob_count)
```

```python
# 3. extend() - Add multiple elements
new_students = ["Mary", "David"]

students.extend(new_students)

print("\nAfter extending the list:")
print(students)


# 4. index() - Find the position of an element
position = students.index("John")

print("\nPosition of John:")
print(position)


# 5. insert() - Insert an element at a specific position
students.insert(1, "Peter")

print("\nAfter inserting Peter:")
print(students)
```

LIST METHODS

Python List pop()

- Removes element at the given index [**list.pop(index)**]

Python List remove()

- Removes item from the list **[list.remove(element)]**

Python List reverse()

- reverses the list [**list.reverse()**]

Python List sort()

- sorts elements of a list [**list.sort(key=..., reverse=...)**]

```python
# Create a list
numbers = [40, 10, 30, 20, 50, 30]

print("Original list:")
print(numbers)


# 1. pop() - Remove an element using its index
removed_item = numbers.pop(2)

print("\nRemoved item:")
print(removed_item)

print("List after pop():")
print(numbers)
```

```python
# 2. remove() - Remove an element by its value
numbers.remove(30)

print("\nList after remove():")
print(numbers)


# 3. reverse() - Reverse the order of the list
numbers.reverse()

print("\nList after reverse():")
print(numbers)


# 4. sort() - Sort the list in ascending order
numbers.sort()

print("\nList after sort():")
print(numbers)
```

EXERCISES

1. What will be the result of the following syntax:

```
mylist = ['apple', 'banana', 'banana', 'cherry']
print(mylist[2]).
```

2. What will be the result of the following syntax:

```
mylist = ['apple', 'banana', 'cherry']
print(mylist[-1])
```

3. What will be the result of the following syntax:

```
mylist = ['apple', 'banana', 'cherry', 'orange', 'kiwi']
print(mylist[1:4])
```

DICTIONARIES

WHAT ARE DICTIONARIES

Dictionaries are data structures that index values by a given key **(key-value pairs)**.

Dictionaries are written with curly brackets **{}**, and they have keys and values.

The general syntax for creating a dictionary is:

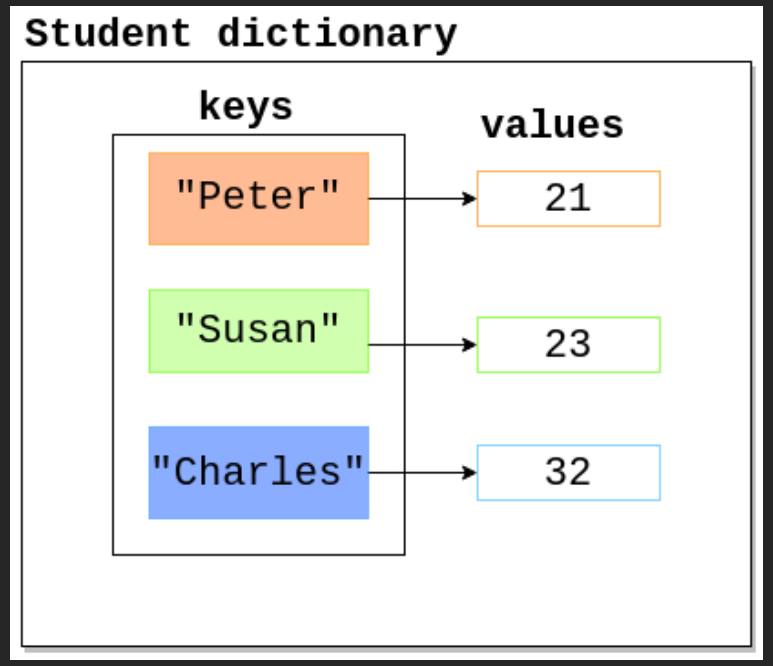
```
DictionaryName {

key1: value1,
key2: value2,
.
.
.
keyN: valueN,
}
```

CREATING A DICTIONARY

Creating an empty dictionary:

- new_dict = dict()
- new_dict = {}



CREATING A DICTIONARY

Creating a dictionary with values:

- *student = {*

```
    "name": "Alice",
    "age": 20,
    "program": "Computer Engineering“
}

print(student)
```

Accessing dictionary values. We use key to access its value

- *student = {*

```
    "name": "Alice",
    "age": 20,
    "program": "Computer Engineering“
}

print(student["name"])
print(student["age"])
```

LOOPING THROUGH THE DICTIONARY

Loop through Keys, by default, a for loop goes through

The dictionary's keys.

```
◦ student = {
        "name": "Alice",
        "age": 20,
        "program": "Computer Engineering“
 }
 for key in student:
      print(key)
```

Loop through values. Use the value() method to loop through the values

```
◦ student = {
        "name": "Alice",
        "age": 20,
        "program": "Computer Engineering“
 }
 for value in student.values():
        print(value)
```

LOOPING THROUGH THE DICTIONARY

Loop Through Keys and Values.

Use the items() method to get both the key and its values.

- *student = {*

  "name": "Alice",

  "age": 20,

  "program": "Computer Engineering“

}

for key, value in student.items():

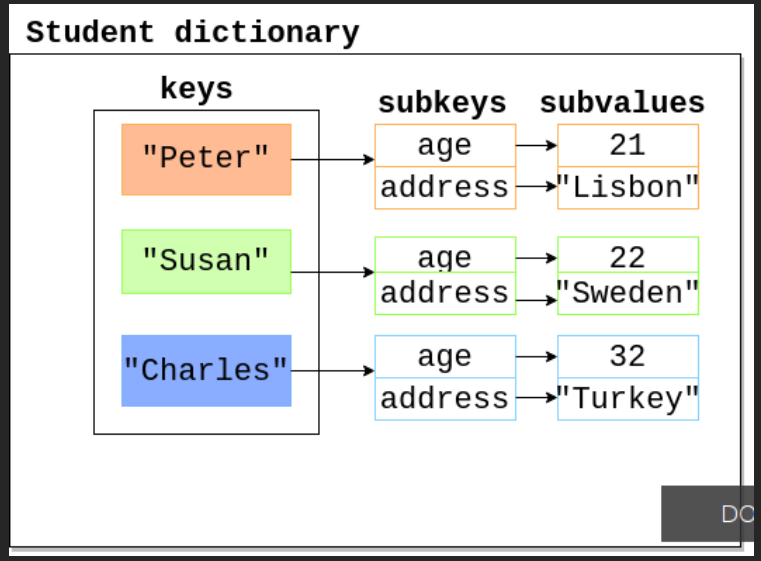
print(key, “:”, value)

```
ages = {
    "Peter": 10,
    "Isabel": 11,
    "Anna": 9,
    "Thomas": 10,
    "Bob": 10,
    "Joseph": 11,
    "Maria": 12,
    "Gabriel": 10,
}
# print one item
print("Get age of peter")
print(ages["Peter"])

## print the whole dictionary
print("Get age of all persons")
for key, value in ages.items():
    print(key, value)
```

NESTED DICTIONARY

A dictionary can be made within a dictionary, and you can also use other dictionaries as values.




```
students = {
    "Peter": {"age": 10, "address": "Lisbon"},
    "Isabel": {"age": 11, "address": "Sesimbra"},
    "Anna": {"age": 9, "address": "Lisbon"},
}
print students
print students['Peter']
print students['Peter']['address']
```

EXERCISES

1. Create and print a dictionary:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict)
```

2. Print the data type of a dictionary:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(type(thisdict))
```

3. Using the dict() method to make a dictionary:

```
thisdict = dict(name = "John", age = 36, country = "Norway")
print(thisdict)
```

EXERCISE

Write a Python program that creates a dictionary containing information about a Year 2 Computer Engineering student, including their name, age, program, year of study, and GPA.

Display the entire dictionary, then use the appropriate dictionary operations to display the student's name and GPA.

Use for loops to display all the keys, all the values, and both keys and values.

Finally, add the student's department to the dictionary and use an if-else statement to check the GPA and display "Excellent Performance" if the GPA is 3.0 or above, otherwise display "Needs Improvement."

PYTHON CONTROLS

- **LOOPS**

LOOPS

What are loops

For Loops

Nested For Loops

While Loops

Exercises

WHAT ARE LOOPS?

A **loop** is a control structure that is used to perform a set of instructions for a specific number of times.

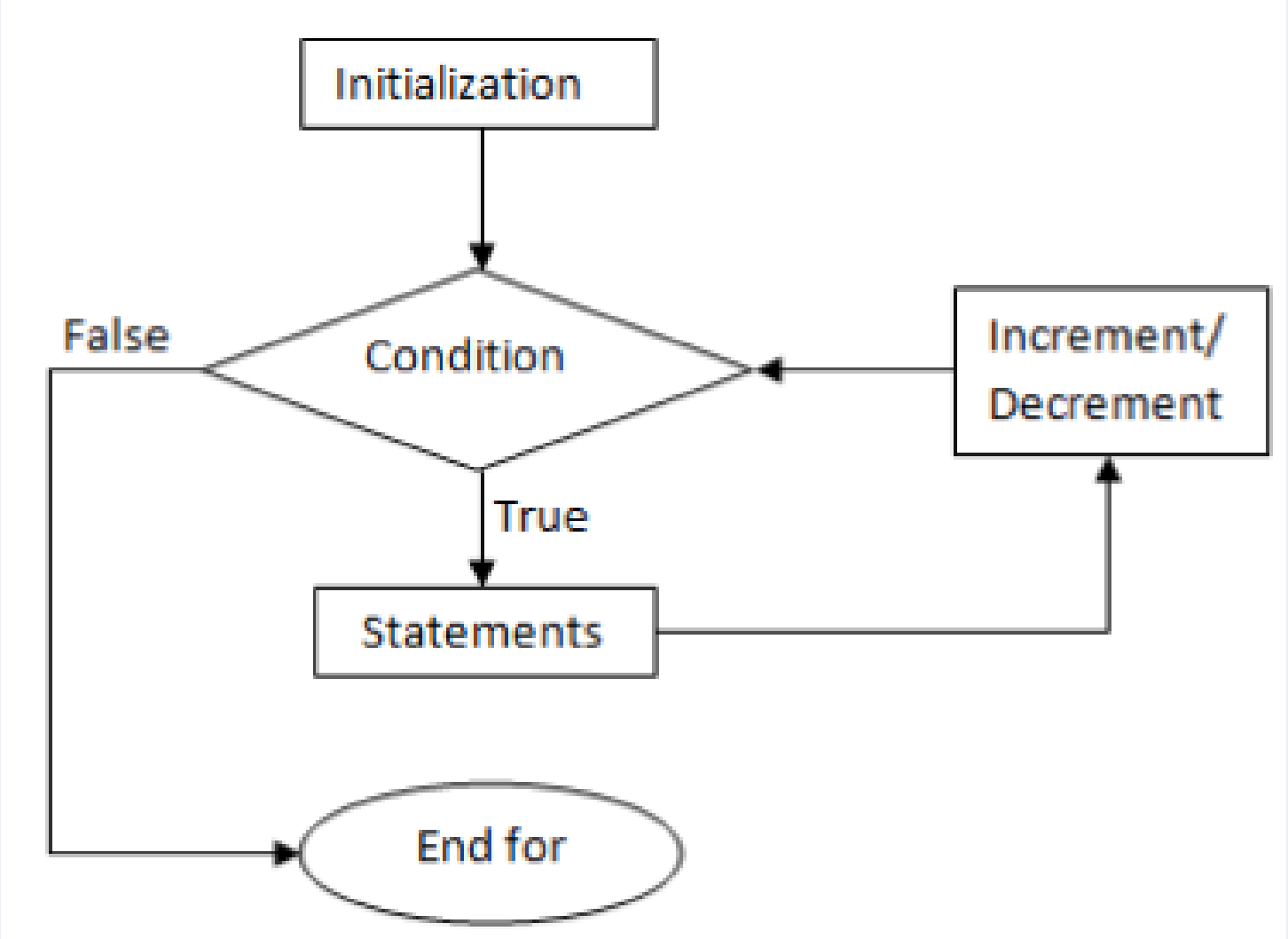
DEFINITION

- Loops solve the problem of having to write the same set of instructions repeatedly.
- Just like conditional statements, a loop is classified as a control structure because it directs the flow of a program by making varying decisions in its **iterations**
- Loops are a crucial part of many popular programming languages such as C++, Java, and JavaScript.

LOOPS IN PYTHON

There are two types of loops that we can use in Python:

- The for loop
- The while loop



THE FOR LOOP

A for loop uses an **iterator** to traverse a sequence

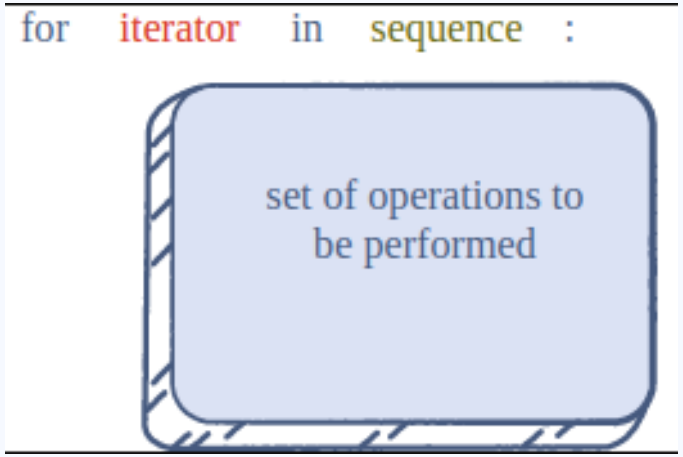
The iterator starts from the beginning of the sequence.

In each iteration, the iterator updates to the next value in the sequence.

The loop ends when the iterator reaches the end.

Structure:

- The name of the iterator
- The sequence to be traversed
- The set of operations to perform



LOOPING THROUGH A RANGE

In Python, the built-in **range()** function can be used to create a sequence of integers

```
range(start, end, step)
```

The **end** value is not included in the list.

If the **start** index is not specified, its default value is **0**

The optional **step** decides the number of steps the iterator jumps ahead after each iteration.

if we don't specify it, the default step is 1

EXAMPLES

1. Sequence of numbers from 1 to 10
2. A sequence of even numbers between 0 and 20 (inclusive)
3. A sequence of numbers from 1 to 10 with a step of 3
4. Print individual letters of a string "PYTHON"
5. Given the list numbers = [2, 7,10, 16, 23, 54] print a list of square of each number in the list
6. Given the float_list = [2.5, 16.42, 10.77, 8.3, 34.21] print how many numbers are less than 10.

NESTED FOR LOOPS

"Python lets us easily create loops within loops"

EXECUTION OF NESTED LOOPS

The inner loop will always complete before the outer loop.

Example: Suppose we want to print two elements whose sum is equal to a certain number **n**

Any Suggestion of the algorithm to use ??

SOLUTION

```
n = 50
num_list = [10, 4, 23, 6, 18, 27, 47]

for n1 in num_list:
    for n2 in num_list:  # Now we have two iterators
        if(n1 + n2 == n):
            print(n1, n2)
```

THE BREAK KEYWORD

Sometimes, we need to exit the loop before it reaches the end.

This can happen if we have found what we were looking for and don't need to make any more computations in the loop.

That's what the **break** keyword is for. It can *break* the loop whenever we want.

```
num_list = [10, 4, 23, 6, 18, 27, 47]
for n in num_list:
    if n < 10:
        print(n)
        break
```

THE CONTINUE KEYWORD

When the **continue** keyword is used, the rest of that particular iteration is skipped.

The loop *continues* on to the next iteration.

```
num_list = list(range(0, 10))

for num in num_list:
    if num == 3 or num == 6 or num == 8:
        continue
    print(num)
```

THE PASS KEYWORD

In all practical meaning, the **pass** statement does nothing to the code execution.

It can be used to represent an area of code that needs to be written.

it is used to assist you when you haven't written a piece of code but still need your entire program to execute

```
num_list = list(range(20))

for num in num_list:
    pass # You can write code here later on

print(len(num_list))
```

THE WHILE LOOP

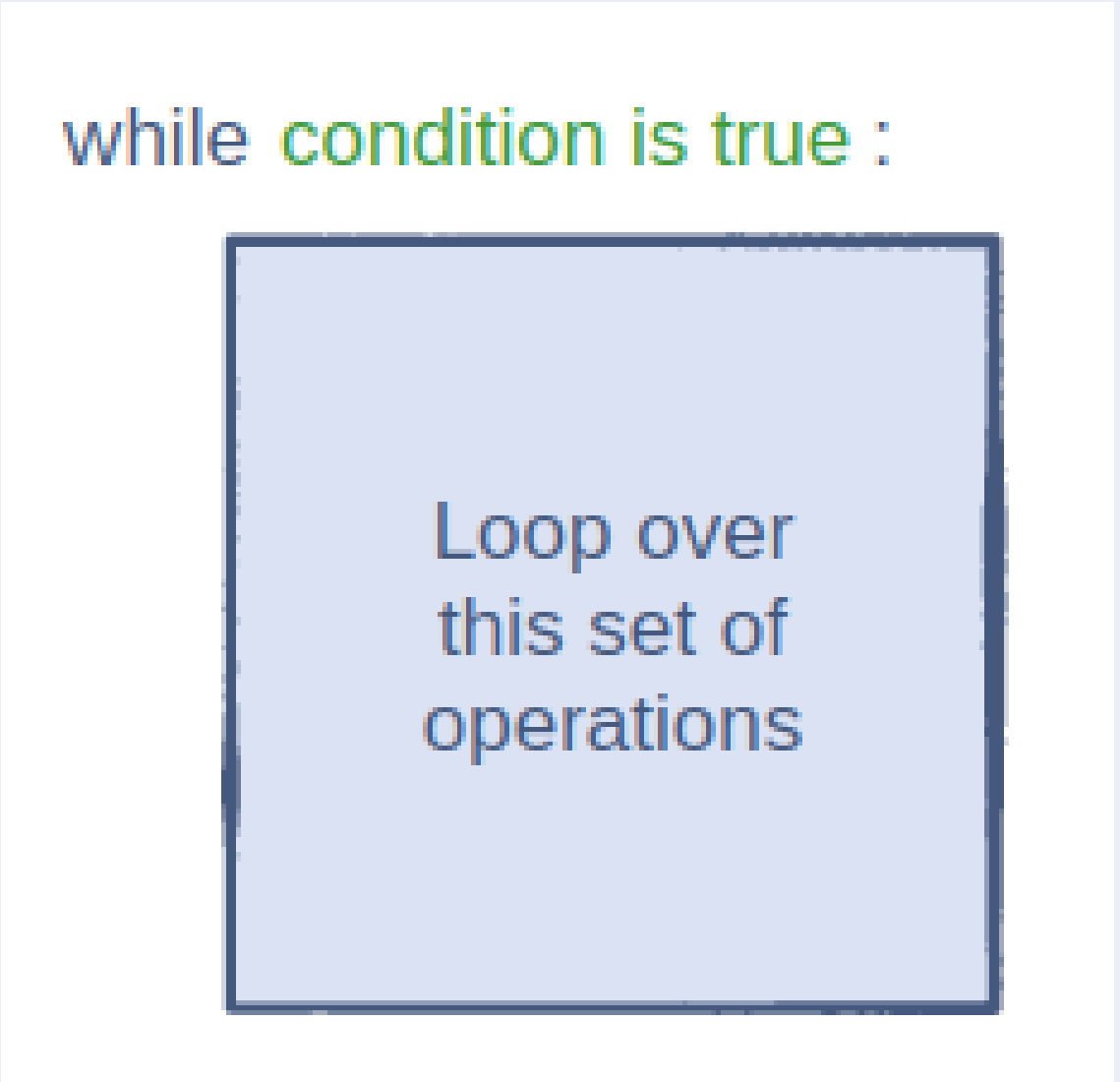
"The while loop keeps iterating over a certain set of operations as long as a certain **condition** holds **True**

While this condition is true, keep the loop running"

WHILE LOOP STRUCTURE

while loop is not always restricted to a fixed range

Its execution is based solely on the condition associated with it.



THE WHILE LOOP IN ACTION

Here's a while loop that finds out the maximum power of n before the value exceeds 1000

```
n = 2  # Could be any number
power = 0
val = n
while val < 1000:
    power += 1
    val *= n
print(power)
```

CAUTIONARY MEASURES

Compared to for loops, we should be more careful when creating while loops.

This is because a while loop has the potential to never end. This could crash a program!

```
1  while(True):
2      print("Hello World")
3
4  x = 1
5  while(x > 0):
6      x += 5
7
```

EXERCISES

1. Create a loop that counts from 0 to 100
2. Make a multiplication table using a loop
3. Output the numbers 1 to 10 backward using a loop
4. Create a loop that counts all even numbers to 10
5. Create a loop that sums the numbers from 100 to 200

FUNCTIONS IN PYTHON

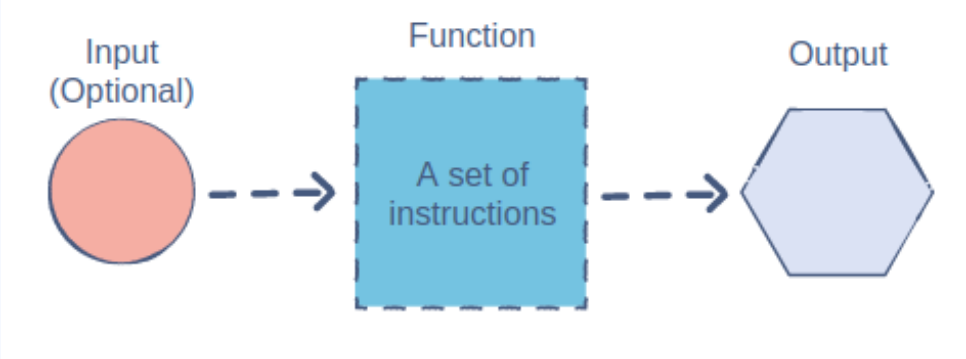
A function is a reusable set of operations

WHY FUNCTIONS?

Functions are useful because they make the code concise and simple.

The primary benefits of using functions are:

- **Reusability**: A function can be used over and over again. You do not have to write redundant code.
- **Simplicity**: Functions are easy to use and make the code readable. We only need to know the inputs and the purpose of the function without focusing on the inner workings.



EXAMPLE

Suppose we want to find the smaller value between two integers.

The good news is that Python already has the min() function:

```
minimum = min(10, 40)
print(minimum)

minimum = min(10, 100, 1, 1000)  # It even works with multiple arguments
print(minimum)

minimum = min("Superman", "Batman")  # And with different data types
print(minimum)

```

```
num1 = 10
num2 = 40
if num1 < num2:
    minimum = num1
else:
    minimum = num2
print(minimum)

num1 = 250
num2 = 120
if num1 < num2:
    minimum = num1
else:
    minimum = num2
print(minimum)

num1 = 100
num2 = 100
if num1 < num2:
    minimum = num1
else:
    minimum = num2
print(minimum)

```

TYPES OF FUNCTIONS IN PYTHON

There are two basic types of functions in Python:

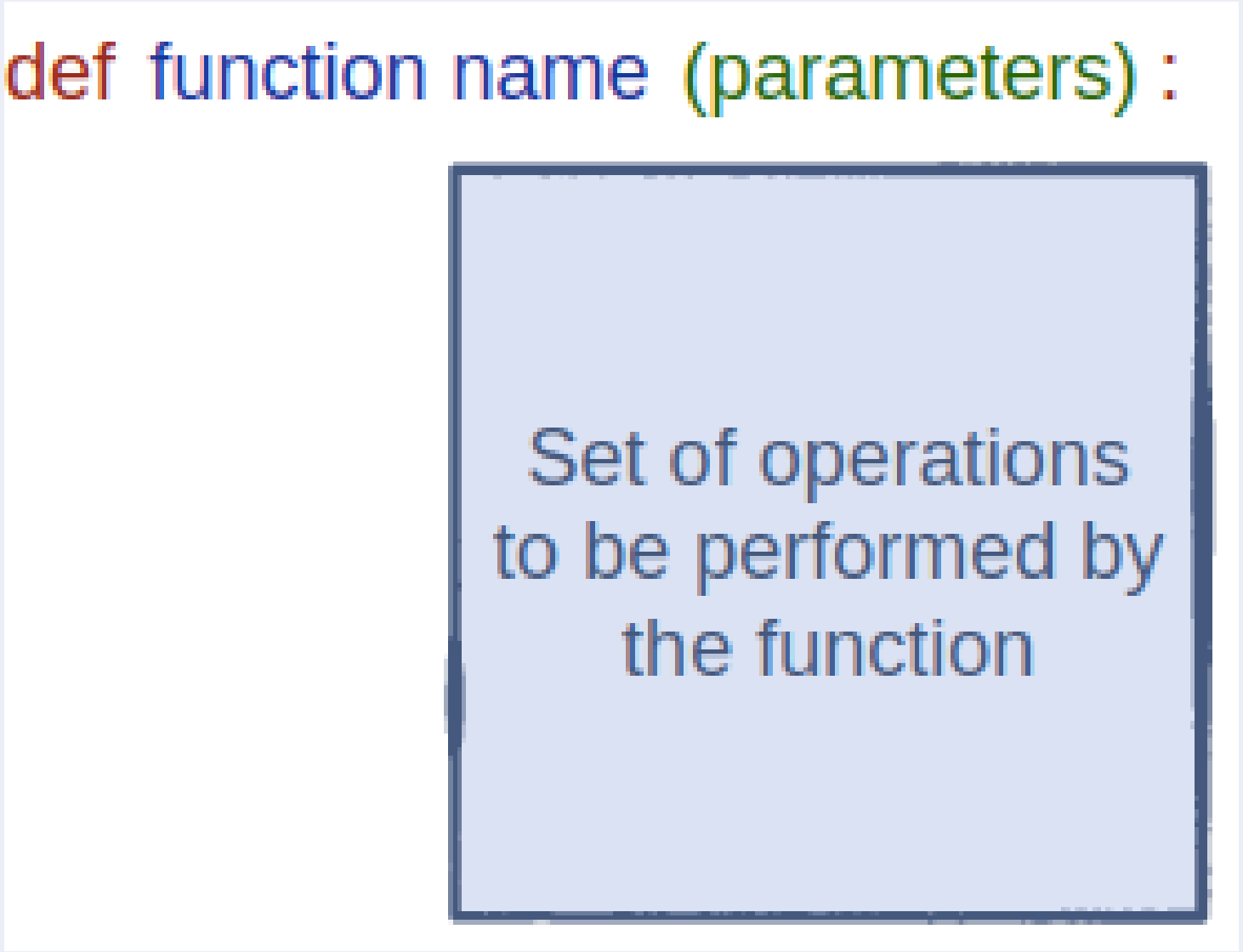
- **Built-in functions**: len(), min(), print(),....
- **User-defined functions**

COMPONENTS OF A FUNCTION

In Python, a function can be *defined* using the **def** keyword

The function **name** is simply the name we'll use to identify the function.

The **parameters** of a function are the inputs for that function. We can use these inputs within the function.



IMPLEMENTATION

Let's start by making a plain function that prints four lines of text.

We can call the function in our code using its name along with empty parentheses.

```
def my_print_function():  # No parameters
    print("This")
    print("is")
    print("A")
    print("function")
# Function ended


# Calling the function in the program multiple times
my_print_function()
my_print_function()
```

FUNCTION PARAMETERS

Parameters are a crucial part of the function structure

They are the means of passing data to the function. This data can be used by the function to perform a meaningful task

The actual values/variables passed into the parameters are known as **arguments**

```
def minimum(first, second):
    if (first < second):
        print(first)
    else:
        print(second)


num1 = 10
num2 = 20

minimum(num1, num2)
```

THE RETURN STATEMENT

To return something from a function, we must use the return keyword

Keep in mind that once the return statement is executed, the compiler ends the function

Any remaining lines of code after the return statement will not be executed.

```
def minimum(first, second):
    if (first < second):
        return first
    else:
        return second


num1 = 10
num2 = 20

result = minimum(num1, num2)  # Storing the value returned by the function
print(result)
```

EXAMPLES & EXERCISES

1. Write a Python function that accepts a string and calculate the number of upper-case letters and lower-case letters

Sample string: PythOn

Number of Uppercase letters: 2

Number of Lowercase letters: 4

2. Write a function that returns the sum of multiples of 3 and 5 between 0 and **limit** (parameter). For example, if limit is 20, it should return the sum of 3, 5, 6, 9, 10, 12, 15, 18, 20.

3. Write a Python function to sum all the numbers in a list.
Sample List : (8, 2, 3, 0, 7)
Expected Output: 20

EXAMPLES & EXERCISES

Exercise 1: Shopping Bill Calculator

Write a Python function called calculate_total() that accepts the price of an item and the quantity purchased. The function should calculate and return the total cost of the items. For example, if a customer buys 4 notebooks at 1,500 RWF each, the function should return 6,000 RWF.

Exercise 2: Student Marks and Grades

Write a Python function called calculate_grade() that accepts a student's marks out of 100 and returns the corresponding grade according to the following rules: marks of 80–100 correspond to grade A, 70–79 to grade B, 60–69 to grade C, 50–59 to grade D, and marks below 50 to grade F. The function should also handle invalid marks outside the range of 0–100.

Exercise 3: Temperature Conversion

Write a Python function called celsius_to_fahrenheit() that accepts a temperature in degrees Celsius and returns the equivalent temperature in Fahrenheit. The conversion formula is \(F = (C \times 9/5) + 32\). For example, if the temperature is 25°C, the function should return 77°F.

EXAMPLES & EXERCISES

Exercise 4: Mobile Money Transaction Fee

Write a Python function called calculate_transaction_fee() that accepts the amount of money a customer wants to transfer using a mobile money service. The function should calculate and return a transaction fee equal to 1% of the amount transferred. For example, if a customer transfers 20,000 RWF, the function should return a fee of 200 RWF.

Exercise 5: Employee Salary Calculator

Write a Python function called calculate_salary() that accepts an employee's basic monthly salary and the amount of any bonus received. The function should calculate and return the employee's total salary before deductions. For example, if the basic salary is 350,000 RWF and the bonus is 50,000 RWF, the function should return 400,000 RWF.

Exercise 6: Electricity Bill Calculator

Write a Python function called calculate_electricity_bill() that accepts the number of electricity units consumed by a household and the price per unit. The function should calculate and return the total electricity cost. For example, if a household consumes 120 units at 250 RWF per unit, the function should return 30,000 RWF.

EXAMPLES & EXERCISES

Exercise 7: Age Calculator

Write a Python function called calculate_age() that accepts a person's birth year and the current year. The function should calculate and return the person's age in years. For example, if a person was born in 2000 and the current year is 2026, the function should return 26. Assume the birthday has already occurred in the current year.

Exercise 8: Restaurant Bill and Tip

Write a Python function called calculate_restaurant_bill() that accepts the total restaurant bill and a tip percentage. The function should calculate and return the final amount to be paid, including the tip. For example, if the bill is 30,000 RWF and the tip is 10%, the function should return 33,000 RWF.

EXAMPLES & EXERCISES

Exercise 9: Fuel Consumption for a Journey

Write a Python function called calculate_fuel_cost() that accepts the distance of a journey in kilometres, a vehicle's fuel consumption in litres per 100 kilometres, and the price of one litre of fuel. The function should calculate and return the total fuel cost for the journey. For example, if a journey is 200 km, the vehicle consumes 8 litres per 100 km, and fuel costs 1,500 RWF per litre, the function should return 24,000 RWF.

Exercise 10: Shopping Discount Calculator

Write a Python function called calculate_discounted_price() that accepts the original price of a product and a discount percentage. The function should calculate and return the final price after applying the discount. For example, if a product costs 50,000 RWF and has a 20% discount, the function should return 40,000 RWF.

OOP IN PYTHON

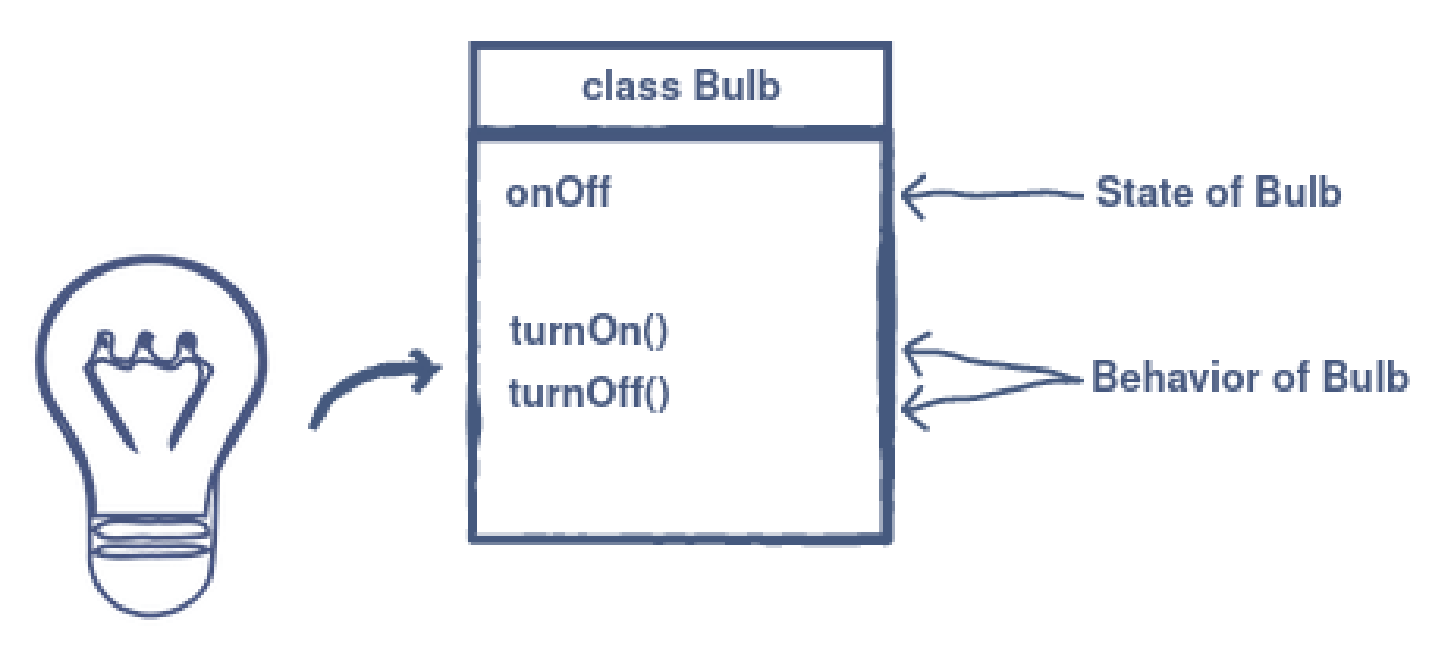
It is a programming paradigm that includes, or relies, on the concept of classes and objects.

ANATOMY OF OBJECTS AND CLASSES

The basic idea of OOP is to divide a sophisticated program into a number of **objects** that talk to each other.

Objects are a collection of **data** and their **behaviors**.

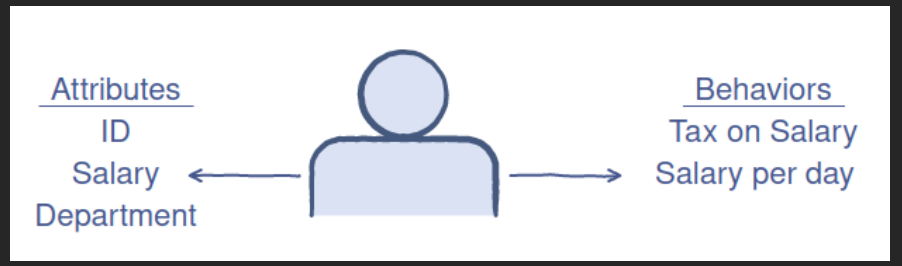
A **class** can be thought of as a *blueprint* for creating objects.

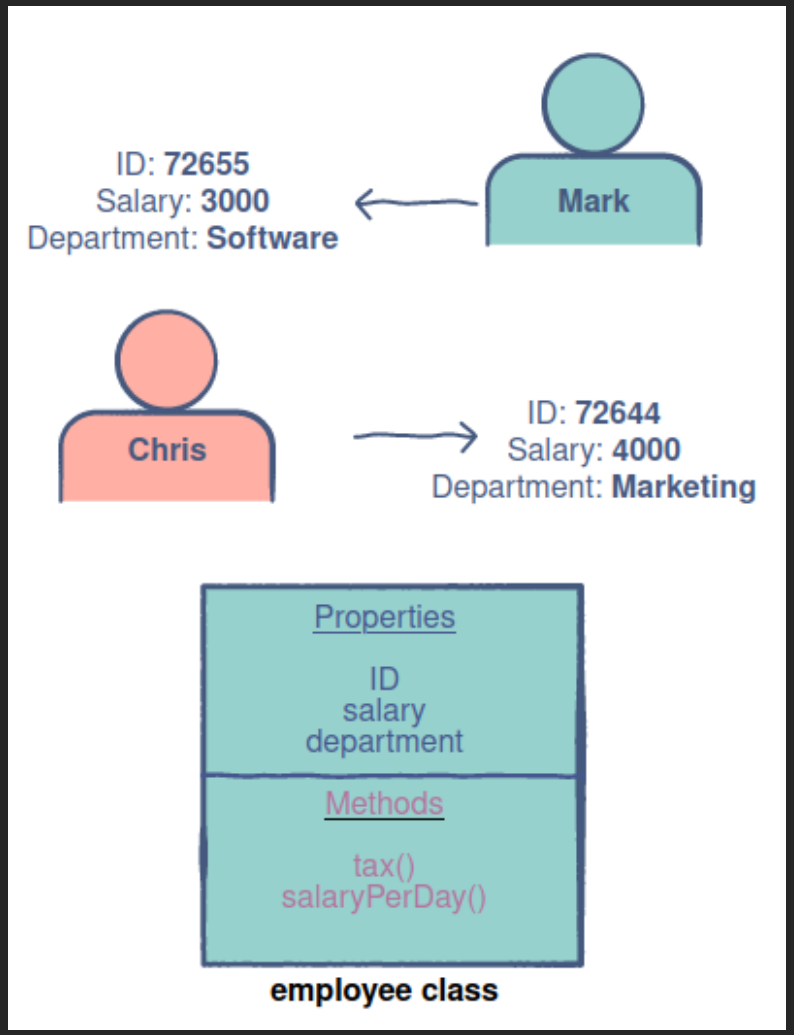


INTRODUCTION TO OBJECTS AND CLASSES

Attributes are also referred to as properties or members. For consistency, we will be using properties throughout the course.

Behaviors are also referred to as member functions or methods. For consistency, we will be using methods throughout the course.





DECLARING A CLASS IN PYTHON

Must start with a *letter* or *underscore*

Should only be comprised of *numbers*, *letters*, or *underscores*

```
1  class MyClass:
2      pass
3
4
5  obj = MyClass()  # creating a MyClass Object
6  print(obj)
7
```

IMPLEMENTATION OF THE EMPLOYEE CLASS

To access properties of an object, the **dot** notation is used.

```
1   class Employee:
2       id = None
3       salary = None
4       department = None
5
6
7   james = Employee()
8
9   james.id = 12345
10  james.salary = 250000
11  james.department = "IT"
12
13  print(james.department)
14
```

INITIALIZING OBJECTS

the **initializer (__init__)** is used to *initialize* an object of a class.

It's a special method that outlines the steps performed when a class object is created in the program.

It's used to define and assign values to **instance variables.**

The initializer is a special method because it **does not have a return type**.

The first parameter of **__init__** is **self**,

```
class Employee:
    def __init__(self, id, salary, department=None):
        self.id = id
        self.salary = salary
        self.department = department

james = Employee(12345, 250000)
print(james.salary)
```

USE OF CLASS VARIABLES

The **class variables** are shared by all instances or objects of the classes.

The **instance variables** are unique to each instance or object of the class.

Class variables are defined **outside** the initializer and instance variables are defined inside the initializer.

```
class Player:
    formerTeams = []  # class variables
    teamName = 'Liverpool'
    def __init__(self, name):
        self.name = name  # creating instance variables


p1 = Player('Mark')
p2 = Player('Steve')

p1 = Player('Mark')
p1.formerTeams.append('Barcelona') # wrong use of class variabl
p2 = Player('Steve')
p2.formerTeams.append('Chelsea') # wrong use of class variable

print("Name:", p1.name)
print("Team Name:", p1.teamName)
print(p1.formerTeams)
print("Name:", p2.name)
print("Team Name:", p2.teamName)
print(p2.formerTeams)
```

```
1  class Player:
2      teamName = 'Liverpool'  # class variables
3
4      def __init__(self, name):
5          self.name = name  # creating instance variables
6          self.formerTeams = []
7
8
9  p1 = Player('Mark')
10 p1.formerTeams.append('Barcelona')
11 p2 = Player('Steve')
12 p2.formerTeams.append('Chelsea')
13
14 print("Name:", p1.name)
15 print("Team Name:", p1.teamName)
16 print(p1.formerTeams)
17 print("Name:", p2.name)
18 print("Team Name:", p2.teamName)
19 print(p2.formerTeams)
```

IMPLEMENTING METHODS IN A CLASS

A **method** is a group of statements that performs some operations and may or may not return a result. The methods can either alter the content of the properties or use their values to perform a particular computation.

There are three types of methods in Python:

instance methods (mostly used in Python OOP)

> Operate on individual objects (instances). They receive **self** as the first parameter and can access or modify the object's attributes.

class methods

> Operate on the class itself. They receive **cls** as the first parameter and are declared using the **@classmethod** decorator.

static methods

> Perform related operations without automatically receiving the instance (self) or class (cls) as the first parameter. They are declared using the **@staticmethod** decorator.

IMPLEMENTING METHODS IN A CLASS

```python
class Student:
    school = "University of Rwanda"

    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    # 1. Instance method
    def get_result(self):
        return self.name, self.marks

    # 2. Class method
    @classmethod
    def get_school(cls):
        return cls.school
```

```python
    # 3. Static method
    @staticmethod
    def is_pass(marks):
        return marks >= 50

student1 = Student("Eric", 25)

print(student1.get_result())
print(Student.get_school())
print(Student.is_pass(25))
```

IMPLEMENTING METHODS IN A CLASS

```
class Employee:
    def __init__(self, id=None, salary=None, department=None):
        self.id = id
        self.salary = salary
        self.department = department


    def tax(self):
        return self.salary * 0.3

    def netSalary(self):
        return self.salary - self.tax()

james = Employee(12345, 250000)
print(james.netSalary())
```

ACCESS MODIFIERS IN PYTHON

In python, access modifiers determine how attributes and methods of a class are accessed.

Python uses public, protected, and private naming conventions to control access to class members.

Public attributes/members can be accessed from inside and outside the class.

Protected attributes/members are intended for internal use within a class and its subclasses. Begin with a single underscore (_)

Private attributes cannot be accessed directly from outside the class but can be accessed from inside the class. Begin with a single underscore (__)

By default, attributes and methods are public in python.

```python
class Employee:
    def __init__(self, name, position, salary, tax):
        self.name = name                # Public attribute
        self._position = position       # Protected attribute
        self.__tax = tax                # Private attribute
        self.__salary = salary          # Private attribute

    # Public method
    def getPosition(self):
        return self._position

    # Public method
    def netSalary(self):
        return self.__salary - self.__tax
```

```python
# Create an Employee object
emp1 = Employee("Alice", "Manager", 500000, 50000)

# Access public attribute
print(emp1.name)

# Access protected attribute (possible, but discouraged externally)
print(emp1._position)

# Access private attributes through public methods
print(emp1.getPosition())
print(emp1.netSalary())
```

EXAMPLES & EXERCISES

Exercise 1: Student Management System

A school wants to develop a system for managing students' information, including student ID, name, age, and course. Create a Student class with these attributes and a display_details() method to display student information, then create three student objects and display their details.Exercise 10: Shopping Discount Calculator

Exercise 2: Bank Account Management

A bank wants an application that allows customers to deposit and withdraw money while protecting their account balances from direct modification. Create a BankAccount class with a private __balance attribute and methods for depositing money, withdrawing money, and checking the balance. Create an account object and test different transactions.

EXAMPLES & EXERCISES

Exercise 3: Vehicle Rental System

A vehicle rental company rents cars and motorcycles to customers. Create a parent class called Vehicle with attributes for brand and year, then create Car and Motorcycle subclasses with their own additional attributes. Use inheritance to display the information of each vehicle.

Exercise 4: Supermarket Product Management

A supermarket wants to manage products by recording their product ID, name, price, and quantity in stock. Create a Product class with a method to calculate the total value of each product's stock. Create at least four product objects, store them in a list, and use a loop to display each product and calculate the total value of all products in stock.

Exercise 5: Shape Area Calculator

A mathematics application needs to calculate the areas of rectangles and circles. Create a parent class called Shape with a calculate_area() method, then create Rectangle and Circle subclasses that override this method to calculate their respective areas. Create objects of both subclasses and demonstrate polymorphism by calculating their areas using a loop.

PYTHON MODULES & PACKAGES

allow programmers to organize reusable code into separate files and directories, making programs easier to maintain, manage, and reuse across different projects.

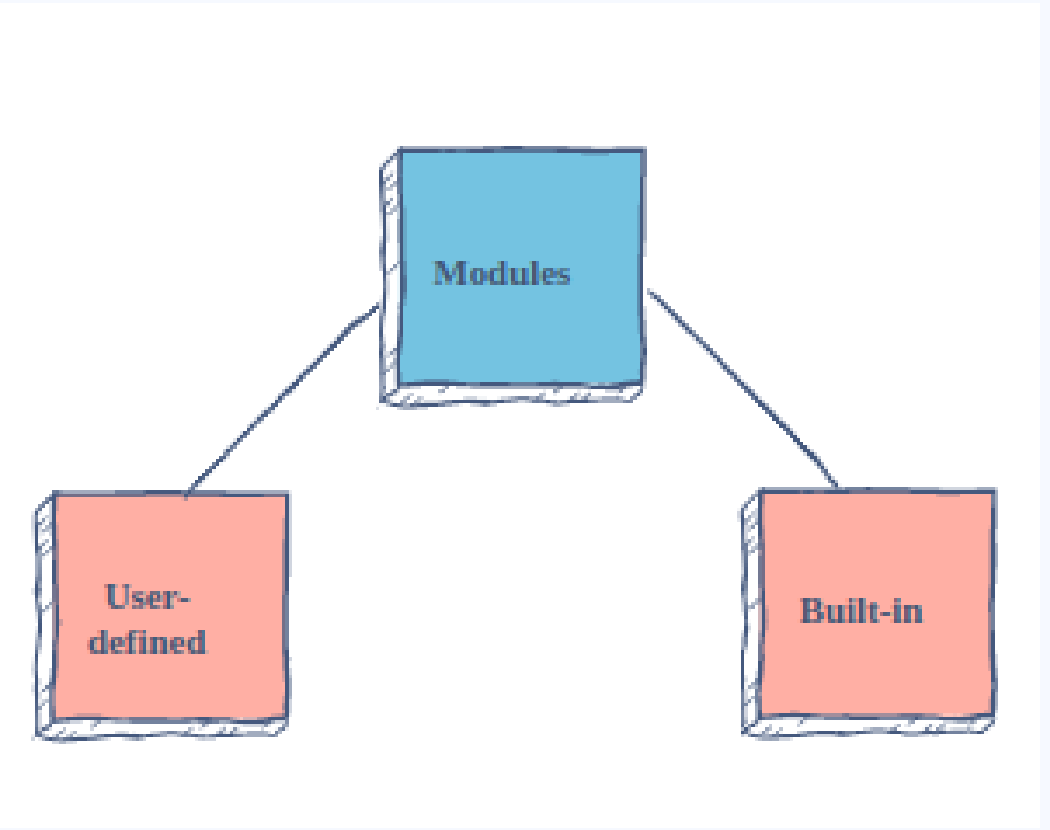
WHAT ARE PYTHON MODULES

A **Python module** is a Python file containing a set of functions and variables to be used in an application.

The variables can be of any type (arrays, dictionaries, objects, etc.)

Types of modules:

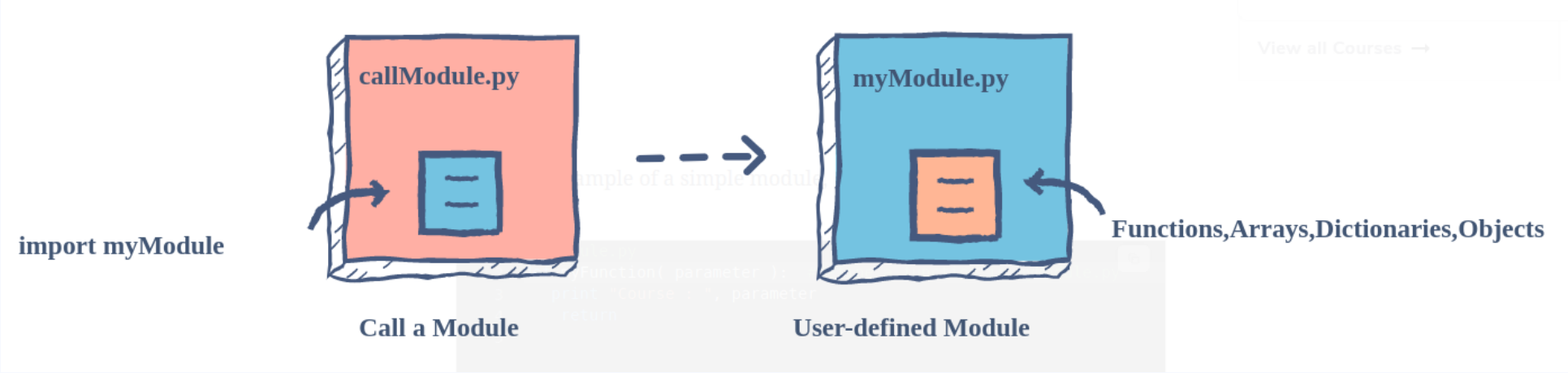
- Built-in
- User defined



USER-DEFINED MODULES

To create a module, create a Python file with a .py extension.

Modules created with a .py extension can be used in another Python source file, using the import statement.



EXAMPLES

Create a module, type the following, and save it as **example.py**.

Python Module addition

```
def add(a, b):
    result = a + b
    return result
```

Import modules in Python

We use the import keyword to do this. To import our previously defined module example, we type the following in the Python prompt.

```
import example
```

- Using the module name we can access the function using the dot . operator.

```
print(example.add(4,5)) # returns 9
```

BUILT-IN MODULES

There are several built-in modules in Python, which you can import whenever you like.

Benefits of Modules in Python

- **Structure Code**
  - Code is logically organized by being grouped into one Python file which makes development easier and less error-prone.
  - Code is easier to understand and use.
- **Reusability**
  - Functionality defined in a single module can be easily reused by other parts of the application.

```
import moduleName #call a module
moduleName.function()#use module function
```

IMPORT MODULES IN PYTHON

To import modules, we use the import statement

- **import module_name**
- **Import module_name.member_name** can be used to import some members(functions) within the module
- **From module_name import *** with this statement we can import all the module members.
- **From module_name import member_name1, member_name2,..** Using the same approach, you can select the members you want to import by separating each with a comma.

EXAMPLES

Import Python Standard Library Modules

The Python standard library contains well **over 200 modules**. We can import a module according to our needs.

Suppose we want to get the value of pi, first we import the math module and use math.pi. For example,

```
# import standard math module
import math


# use math.pi to get value of pi
print("The value of pi is", math.pi)
```

Python import with Renaming

In Python, we can also import a module by renaming it. For example,

```
# import module by renaming it
import math as m


print(m.pi)
```

EXAMPLES

Installing and Using Built-in Modules in Python

- **Built-in modules** are provided with Python and generally do not require installation.

- Import a module using the `import` keyword.

- Examples include `math`, `random`, `datetime`, and `statistics`.

```
import math

import random

print(math.sqrt(25))

print(random.randint(1, 10))
```

A random integer between 1 and 10

EXAMPLES

Python from...import statement

We can import specific names from a module without importing the module as a whole. For example,

```
# import only pi from math module
from math import pi

print(pi)
```

Import all names

In Python, we can import all names(definitions) from a module using the following construct:

```
# import all names from the standard module math
from math import *

print("The value of pi is", pi)
```

PYTHON PACKAGES

A package is a directory used to organize related Python modules and sub-packages.

Traditionally, a regular Python package contains an **__init__.py** file, although modern Python also supports namespace packages without this file.

Python programmers can create their own packages or use packages developed by others.

Python Package Index (PyPI) is an online repository of Python packages.

pip is a tool used to ***install and manage Python packages***.

Packages can be installed system-wide or within a virtual environment, which isolates project dependencies.

Command to install a package:

pip install package-name

BUILT IN PACKAGES

Installing and Using External Python Modules

External modules are developed separately and may need to be installed before use.

Use pip to install modules from the Python Package Index (PyPI).

Import the installed module using the **import keyword.**

Step 1: Install the module

python -m pip install requests

Step 2: Use the module

import requests

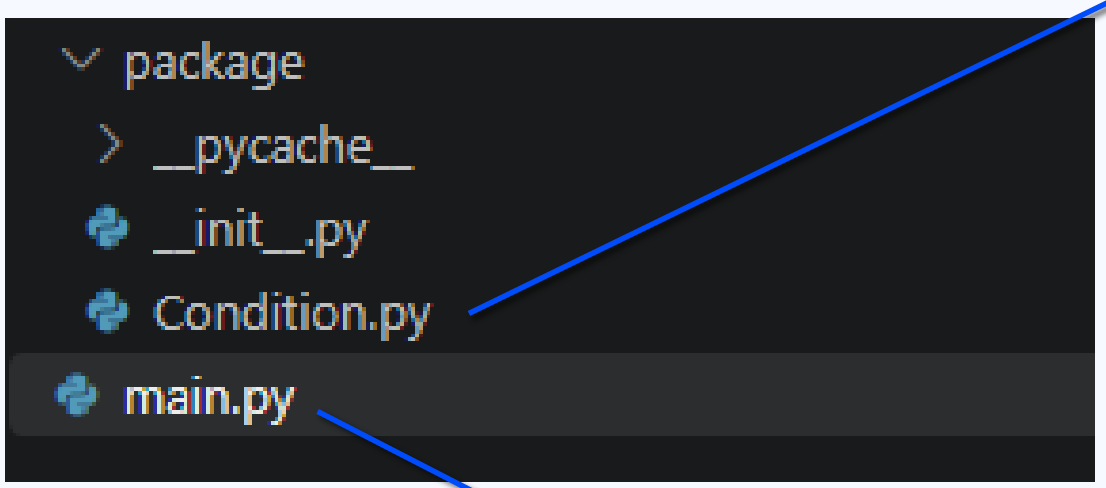
response = requests.get("https://example.com")

print(response.status_code)

Output:

200

USER DEFINED PACKAGE




```python
def add(a, b):
    result = a + b
    return result

def mult(a, b):
    result = a*b
    return result
```

```python
from package.Condition import *

print(add(4,5))
print(mult(30,3))
```

EXAMPLES & EXERCISES

Exercise 1: Student Marks Analysis

Write a Python program that uses the built-in statistics module to calculate the mean and median of a list of students' marks. Display both results.

Exercise 2: Random Password Generator

Write a Python program that uses the built-in random module to select six random characters from a list of letters and digits to generate a simple password.

Exercise 3: Date and Age Calculator Package

Create a user-defined package called date_package containing a module named date_tools.py with a function called calculate_age(birth_year). Use Python's built-in datetime module to obtain the current year and calculate the person's age. Import the function into a separate Python file and display the calculated age.

PYTHON VIRTUAL ENVIRONMENT

A virtual environment is an isolated Python environment that allows each project to manage its own dependencies independently.

Virtualenv is one of the modules used to create Python environments.

Alternatives are **venv, pipenv**, **conda**,....

Installing virtualenv: **pip install virtualenv**

To create a virtual environment with virtualenv: **virtualenv env_name**

After running this command, a directory named env_name will be created, and it will have all the necessary tools Python needs to create a Python environment.

Create a virtual environment in windows.

python –m venv myenv ~~(name of the virtual env)~~

ACTIVATING VIRTUAL ENVIRONMENT

A Python virtual env. Needs to be activated.

In the directory where you create the environment, you can run the following command.

On Windows:

env_name\Scripts\activate

On linux/mac:

source env_name/bin/activate

Once active the name of the environment will be displayed on the left side of your terminal/command line interface

Deactiving a virtual env:

deactivate

Note: Run the pip installation command in the terminal, preferably with the project's virtual environment activated.

EXAMPLES & EXERCISES

Exercise 1: Creating and Managing a Virtual Environment

Create a Python virtual environment named student_env using Python 3.14, activate it, verify the Python version, install the requests package, check that the package is installed, and then deactivate the environment.

Exercise 2: Managing Dependencies for Different Projects

Create two Python projects named project_A and project_B, each with its own virtual environment. Install the requests package in project_A and the numpy package in project_B, then verify that each project has its own dependencies and demonstrate how to activate and deactivate each environment.



THANK YOU